



University of Tehran
Department of Electrical and Computer Engineering

Digital System Design with Hardware Description Languages - Spring 1405
Computer Assignment 3: Getting Started with RTL and BFM Design in SystemC
Due Date: Khordad 26, 1405

Description:

In this assignment, you are to design, implement, and evaluate a sequential division hardware architecture using SystemC. While you have already implemented this architecture in VHDL for Computer Assignment 1, this assignment is explicitly designed to transition those concepts into SystemC RTL design, establish your simulation framework, and validate your local development environment for the higher levels of abstraction and advanced modeling paradigms coming later in the course.

The central objective is the realization of a parameterized sequential restoring division algorithm, split across four distinct phases. Moving beyond RT level hardware implementation, this assignment challenges you to exercise higher-level abstraction modeling through Bus-Functional Modeling, ensuring that the external bus activity and signal traces mirror a real hardware implementation.

Phase 1: Parameterized Unsigned Restoring Division RTL Design

In Phase 1, you are to design and implement a sequential division unit for unsigned integers utilizing the classical restoring division algorithm. To ensure scalability, all modules must be fully parameterized using C++ templates or constructor parameters, allowing the operand bit-width to be configured at instantiation without altering the core logic.

The physical execution flow must be coordinated over multiple clock cycles by a dedicated controller designed under the Huffman coding style, explicitly separating next-state combinational logic, sequential state registers, and combinational output decode structures. The division execution procedure begins with a full pulse on the start signal of the divider, and upon completing the sequential operations over the required bit-width iterations, the ready or done output must be asserted high to signify that the stable quotient and remainder are ready.

Derive the complete RTL schematic of the parameterized datapath for the division core. Design the Huffman controller state diagram for the divider, specifying whether a hybrid Mealy/Moore implementation is required to update register states within a single clock cycle. Implement the

SystemC description for the partitioned, parameterized datapath module. Implement the SystemC description for the Huffman controller module. Construct a testbench to simulate and functionally test the unsigned division core.

Phase 2: Signed Division Extension and Wrapper Design

In Phase 2, you are to extend your parameterized unsigned restoring division core so that the system can seamlessly process signed integer division in two's complement format. This extension must be accomplished without modifying a single line of the internal structural logic of your Phase 1 unsigned computational core.

You will design and implement a signed-operation wrapper module around the unsigned core interface. The wrapper must inherit or pass the configured bit-width parameter. It must intercept incoming signed operands, extract their absolute magnitude to drive the internal unsigned core, and preserve the input sign profiles to perform post-calculation corrections on the final outputs. Both inputs and outputs at the wrapper boundary must be interpreted as signed values.

Design the block diagram for the signed wrapper, detailing the absolute value conversion units, sign-tracking registers if any, and the output sign-correction logic blocks. Write the SystemC description for the complete parameterized signed divider wrapper system. Test functional correctness using representative test cases in a testbench that validates correct signed division behavior across multiple quadrant boundaries of positive/positive, positive/negative, negative/positive, and negative/negative.

Phase 3: Bus-Functional Modeling of the Signed Divider

In Phase 3, you will shift your architectural abstraction level from RTL structural modeling to Bus-Functional Modeling (BFM). You are to model the entire signed division functionality developed in Phase 2 using a transaction-centric BFM approach, maintaining the same configurable bit-width.

Instead of tracking explicit clock-cycle-by-clock-cycle structural register updates, the BFM encapsulates the behavioral timing, handshaking protocols, and communication mechanisms of the complete signed divider at a higher level of abstraction. This model hides the microarchitectural details of the internal restoring shift-and-subtract loop while perfectly mimicking the port-level transaction interfaces, ready-start protocol handshakes, and pin-accurate interfaces of the complete signed hardware system.

Write the SystemC description for the Bus-Functional Model of the entire signed divider system. Test the standalone BFM design using a testbench to ensure functional correctness and protocol adherence through simulation.

Phase 4: Co-Simulation, Comparison, and Trace Alignment

SystemC allows hardware designers to harness advanced C++ software constructs directly within the simulation environment. In Phase 4, you will exploit this capability to construct a dual-model co-simulation testbench to directly compare your implementations.

The final testbench must simultaneously instantiate both Phase 2 RTL structural signed wrapper design and Phase 3 signed Bus-Functional Model, configured with identical bit-widths. The testbench must drive identical, randomized input vectors into both models concurrently. You must utilize C++ standard library random number generators to generate a high volume of random test patterns, covering extreme edge cases like maximum/minimum signed bit-width bounds.

The testbench must dynamically monitor and cross-check the outputs of both models on every simulation clock cycle. From an external perspective, the high-level behavior, handshakes, ready signals, and final computational outputs of the BFM must match the behavioral throughput of the RTL hardware exactly. You must dump all system signals into a single Value Change Dump (VCD) trace file to demonstrate that the external waveform traces from both models align perfectly.

Deliverables:

Your final submission package must be structured cleanly and contain the following two main components:

- A. A complete collection of your design files, including source files, simulation scripts and testbenches and synthesis netlists alongside reports on cell usage, area, and timing.
- B. A comprehensive report using IEEE manuscript templates for conference proceeding. Including design illustration and description, implementation details, simulation results and output waveform justifications.

Attention:

Make sure that you compressed all your source codes and documents in a zip file and named it in this format “YourFirstName-YourLastName-CA#3”.
